Common Table Expressions (CTE) on SQL

By Don Schlichting

This article will introduce Common Table Expressions (CTE) on SQL , and compare them with related methods such as Derived and Temporary Tables.

Introduction

A Common Table Expression is an expression that returns a temporary result set from inside a statement. This result set is similar to a hybrid Derived Table meets declared Temporary Table. The CTE contains elements similar to both. Some of the most frequent uses of a Common Table Expression include creating tables on the fly inside a nested select, and doing recursive queries. Common Table Expressions can be used for both selects and DML statements. The natural question is, if we have been using TSQL for this long without Common Table Expressions, why start using them now? There are several benefits to learning CTEs. Although new to SQL Server, Common Table Expressions are part of ANSI SQL 99, or SQL3. Therefore, if ANSI is important to you, this is a step closer. Best of all, Common Table Expressions provide a powerful way of doing recursive and nested queries in a syntax that is usually easier to code and review than other methods.

CTE

Below is very simple Common Table Expression example. The example CTE will be used to return the list price of products, and our sell price, which is five percent below list. This first example is very simple, and could be replaced by a single select statement, but it will demonstrate some key CTE points. The examples will get more advanced as the article progresses.

```
USE AdventureWorks
GO

WITH MyCTE( ListPrice, SellPrice) AS
(
  SELECT ListPrice, ListPrice * .95
  FROM Production.Product
)

SELECT *
FROM MyCTE

GO
```

The database, Adventure Works, is now the SQL default sample database. Gone are the days of Northwind. On the beta version used for this article, Adventure Works did not install by default. Instead, during install, click Advanced then select sample databases.

The Common Table Expression is created using the WITH statement followed by the CTE name. Immediately trailing is a column list, in our case, we are returning two columns,

ListPrice, and SellPrice. After the AS, the statement used to populate the two returning columns begins. The CTE is then followed by a select calling it.

The BOL format of a Common Table Expression is listed below;

```
[ WITH <common_table_expression> [ ,...n ] ]

<common_table_expression>::=
    expression_name [ ( column_name [ ,...n ] ) ]
    AS
    ( CTE_query_definition )
```

Temporary Tables

This small example displays several interesting concepts. The CTE is called by name from the SELECT * statement, similar to a Temporary Table. However, if a Temporary Table were used, it would first have to be created, and then populated;

```
CREATE TABLE #MyCTE
(
  ListPrice money,
  SellPrice money
)

INSERT INTO #MyCTE
          (ListPrice, SellPrice)
          SELECT ListPrice, ListPrice * .95
          FROM Production.Product
```

However, a Temporary Table could be called over and over again from with in a statement. A Common Table Expression must be called immediately after stating it. Therefore, in this example, the call to the CTE will fail.

```
USE AdventureWorks
GO

WITH MyCTE( ListPrice, SellPrice) AS
(
  SELECT ListPrice, ListPrice * .95
  FROM Production.Product
)

SELECT *
FROM Production.Location

SELECT *
FROM MyCTE

GO
```

The call to MyCTE will fail with the following error:

Msg 208, Level 16, State 1, Line 14
Invalid object name 'MyCTE'.

The CTE itself has some syntactical restrictions. Compute, Order By (without a TOP), INTO, Option, FOR XML, and FOR BROWSE are all not allowed.

Derived Tables

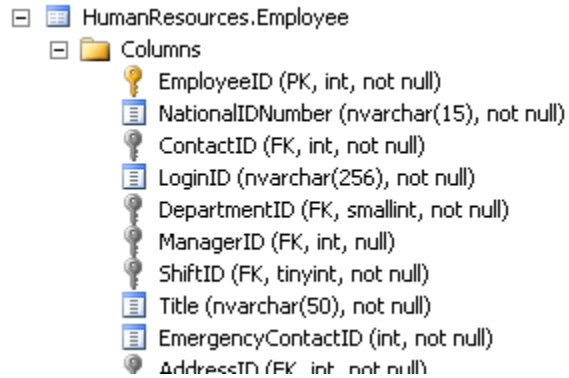
A SQL method somewhat similar to a CTE is a Derived Table. These are temporary result sets created by a query statement. However, they cannot be referenced by name, and may be used only once. In complex statements, they may not be as readable as Common Table Expressions. The CTE rewritten as a Derived Table would be:

```
SELECT *
FROM (
  SELECT ListPrice, (ListPrice * .95) AS SellPrice
  FROM Production.Product
  )
MyDerivedTable
```

Common Table Expressions will not displace Derived Tables however. There will still be many instances when Derived Tables will be a better fit. However, in some complex statements, Common Tables Expression may be clearer to understand.

Recursive Queries

One of the most powerful features of Common Table Expressions is their ability to be self-referenced by name, allowing for a recursive query. A Recursive Query is a query that calls itself. Below is a modified example of BOL recursive query. The purpose of the statement is to display each employee at Adventure Works, and who their manager is. The Employee table is self

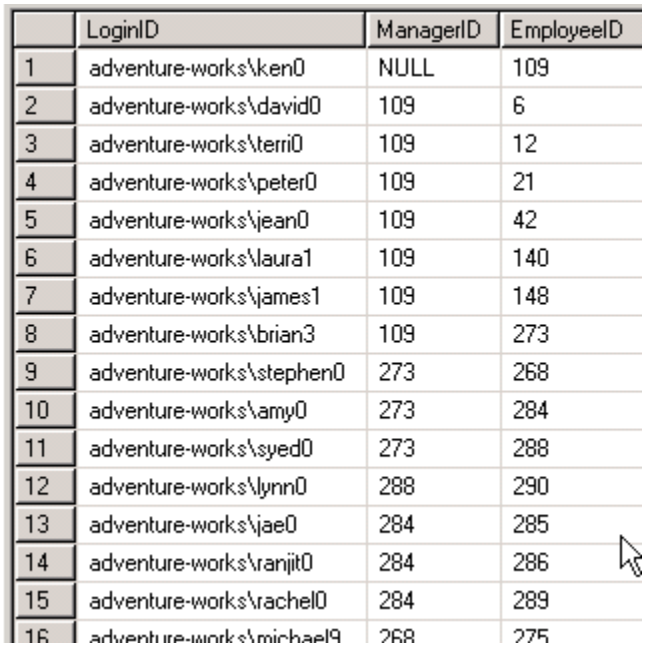


The statement first looks for all top-level managers, the records where the ManagerID is null. In this table, there is only one. Next, we will self-reference this CTE by using its name "DirectReports" joined back to our top-level managers.

```
USE AdventureWorks ;
GO
WITH DirectReports(LoginID, ManagerID, EmployeeID) AS
(
    SELECT LoginID, ManagerID, EmployeeID
    FROM HumanResources.Employee
    WHERE ManagerID IS NULL
    UNION ALL
    SELECT e.LoginID, e.ManagerID, e.EmployeeID
```

```
        FROM HumanResources.Employee e
    INNER JOIN DirectReports d
    ON e.ManagerID = d.EmployeeID
)
SELECT *
FROM DirectReports ;
GO
```

Running this statement produces the following result set:



	LoginID	ManagerID	EmployeeID
1	adventure-works\ken0	NULL	109
2	adventure-works\david0	109	6
3	adventure-works\terri0	109	12
4	adventure-works\peter0	109	21
5	adventure-works\jean0	109	42
6	adventure-works\laura1	109	140
7	adventure-works\james1	109	148
8	adventure-works\brian3	109	273
9	adventure-works\stephen0	273	268
10	adventure-works\amy0	273	284
11	adventure-works\syed0	273	288
12	adventure-works\lynn0	288	290
13	adventure-works\jae0	284	285
14	adventure-works\ranjit0	284	286
15	adventure-works\rachel0	284	289
16	adventure-works\michael9	268	275

The first result line shows Ken, as the top-level manager, reporting to no one. Beneath him reports David, Terri, Peter, Jean, Laura, James and Brian. At Stephen, we see a change, with him reporting to Brian.

The Recursive Common Table Expression introduces some additional syntax. The statement now has two definition components. The first is called the Anchor Member:

```
SELECT LoginID, ManagerID, EmployeeID
    FROM HumanResources.Employee
    WHERE ManagerID IS NULL
```

This statement must be followed by a UNION ALL. The purpose of the UNION ALL is to tie the results of two statements into one result set.

The next definition is called the recursive member. It calls back to itself using the CTE name; *INNER JOIN DirectReports d*

MAXRECURSION

The key word MAXRECURSION can be used as a query hint to stop a statement after a defined number of loops. This can stop a CTE from going into an infinite loop on a poorly coded statement. To use it in our above example, include it on the last line;

```
SELECT *
FROM DirectReports;
OPTION (MAXRECURSION 3);
```

Conclusion

Common Table Expressions will be worth exploring for anyone doing TSQL work. Their use in Recursive Queries alone warrants them worthy of consideration. Combined with their straightforward syntax, expect to see them used frequently in complex statements.